

Programming SQL Server 2016

Retrieving data from database using T-SQL

Topics

- Retrieving data (SELECT statement)
- "USE" statement
- Simple select
- Changing column heads
- Limiting rows
- Sorting
- Filtering
- Filtering: Using LIKE operator
- Scalars
- Aggregate Functions
- Union

Homework

- All the queries are based on Northwind sample database
1. Show customers who live in USA, UK and Germany along with CustomerId, CompanyName, and Country. And also sort the resultset by Country in Descending order
 2. Show customers who live in USA, UK and Germany along with CustomerId, CompanyName, and Country. But you want to change column heads – Customer Code for CustomerID column, 'Company' for CompanyName column and 'Origin' for Country column.
 3. Show order information from Orders table for orders placed between date January 01, 1997 and December 31, 2000.
 4. Show customers' information from Customers table only for those customers whose CompanyName starts with letter 'A'.
 5. Show customers' information from Customers table only for those customers whose CompanyName starts with letter 'A', 'F' or 'W'.
 6. Show the latest 3 orders from Orders table.
 7. Show the oldest 3 orders from Orders table

Important points

-
-
-

Summary

-
-
-

SELECT Statement

- SELECT statement retrieves rows from the database and enables the selection of one or many rows of columns from one or many tables.
- The main clauses can be summarized as:
`SELECT select_list FROM table_source`
`WHERE search_condition`
`GROUP BY group_by_expression`
`HAVING search_condition`
`ORDER BY order_expression`
- Only SELECT clause is always required and other clauses you add for various purposes
- The order of the clauses is significant. When the optional clauses are used, they must appear in the appropriate order.

The "USE" statement

- Tables hold data, tables are contained in database.
- So, to retrieve data from a table, you have to set your current working database appropriate one
- The "USE" statement sets the current database
- In our lessons, we will use Northwind database
- So, the first statement, we have to run is:
`USE Northwind`
`GO`

Start writing SELECT Query

- Let's try some simple SELECT query
- For more of the examples we will use Northwind sample database
- If you do not have Northwind database, you have to install it.
- In attachments there is a file "insnwind2k12.sql" file Open the inQuery Editor of SQL Management studio, execute the whole script.

Simple SELECT

- Run the first statement:
`USE Northwind`
`GO`

Select all rows including all columns

```
SELECT * FROM customers
GO
```

- The result should be like below

CustomerID	CompanyName	..	Fax
ALFKI	Alfreds Futterkiste	..	030-0076545
ANATR	Ana Trujillo ..	..	(5) 555-3745
..	..	..	..

- The statement returns all rows and includes all columns
- * is a wild card, that stands for all columns
- The columns are shown in natural order - as they are in the table

Selecting with column list

```
SELECT CustomerID, Country, City
FROM Customers
GO
```

- The statement returns all rows and includes columns those are listed in column list
- The result should be like below

CustomerID	CompanyName	Country	City
ALFKI	Alfreds Futterkiste	Germany	Berlin

ANATR	Ana Trujillo Emparedados y helados	Mexico	México D.F.
ANTON	Antonio Moreno Taquería	Mexico	México D.F.
AROUT	Around the Horn	UK	London
..	..	..	..

Changing column Heads

- To change column heads, you have three syntax
→ column-name AS 'Column Head'
→ column-name 'Column Head'
→ 'Column Head' = column-name

Try this

```
SELECT CustomerID as 'Customer Code',
CompanyName 'Company',
'Contact'=Phone
FROM Customers
GO
```

- The result

Customer Code	Company	Contact
ALFKI	Alfreds Futterkiste	030-0074321
ANATR	Ana Trujillo Emparedados y helados	(5) 555-4729
ANTON	Antonio Moreno Taquería	(5) 555-3932
..	..	..

Aliasing tables

- You can use explicit table reference with column in select statement
- For example
`SELECT Customers.CustomerID, Customers.Country, Customers.City`
`FROM Customers`
`GO`
- The readability of a SELECT statement can be improved by giving a table an alias, also known as a correlation name or range variable.
- A table alias can be assigned either with or without the AS keyword:
 - table-name AS table-alias
 - table-name table-alias

Try this

```
SELECT c.CustomerID, c.Country,
c.City
FROM Customers c
GO
```

- If an alias is assigned to a table, all explicit references to the table in the Transact-SQL statement must use the alias, not the table name.
- For example, the following SELECT generates a syntax error because it uses the name of the table when an alias has been assigned:
`SELECT Customers.CustomerID, Customers.Country, Customers.City`
`FROM Customers`
`GO`
--fails

Using TOP clause

- TOP clause limit the number of rows returned
- Syntax
- SELECT TOP n column-list FROM table-name
- returns top n rows of the result-set
- SELECT TOP n PERCENT column-list FROM table-name
- returns top n% rows of the result-set
- use whole number in place of n

```
SELECT TOP 3 CustomerID, Country
FROM customers
GO
```

- The result

CustomerID	Country
ALFKI	Germany
ANATR	Mexico
ANTON	Mexico

```
SELECT TOP 3 PERCENT CustomerID, Country
FROM customers
GO
```

- The result

CustomerID	Country
ALFKI	Germany
ANATR	Mexico
ANTON	Mexico

Sorting Query Results

- To sort ORDER BY Clause is used
 - Syntax is:
- ```
SELECT column-list FROM table-name
ORDER BY column ASC | DESC, column ASC | DESC, ...
```
- ASC for ascending order, DESC from descending order

### Try this

```
SELECT CustomerID, CompanyName, Country, City
FROM Customers
ORDER BY Country ASC
GO
```

- The result

| CustomerID | CompanyName                | Country   | City         |
|------------|----------------------------|-----------|--------------|
| CACTU      | Cactus Comidas para llevar | Argentina | Buenos Aires |
| OCEAN      | Océano Atlántico Ltda.     | Argentina | Buenos Aires |
| RANCH      | Rancho grande              | Argentina | Buenos Aires |
| ERNSH      | Ernst Handel               | Austria   | Graz         |
| PICCO      | Piccolo und mehr           | Austria   | Salzburg     |
| ..         | ...                        | ...       | ..           |

- For Descending Sorting use DESC option

### Try this

```
SELECT CustomerID, CompanyName, Country, City
FROM Customers
ORDER BY customerid DESC
GO
```

- The result

| CustomerID | CompanyName          | Country | City     |
|------------|----------------------|---------|----------|
| WOLZA      | Wolski Zajazd        | Poland  | Warszawa |
| WILMK      | Wilman Kala          | Finland | Helsinki |
| WHITC      | White Clover Markets | USA     | Seattle  |
| ...        | ...                  | ...     | ...      |

- You can set sort order on multiple column

### Try this

```
SELECT CustomerID, CompanyName, Country, City
FROM Customers
ORDER BY Country ASC, City DESC
GO
```

- The result

| CustomerID | CompanyName                | Country   | City         |
|------------|----------------------------|-----------|--------------|
| CACTU      | Cactus Comidas para llevar | Argentina | Buenos Aires |
| OCEAN      | Océano Atlántico Ltda.     | Argentina | Buenos Aires |
| RANCH      | Rancho grande              | Argentina | Buenos Aires |
| PICCO      | Piccolo und mehr           | Austria   | Salzburg     |
| ERNSH      | Ernst Handel               | Austria   | Graz         |
| ..         | ...                        | ...       | ...          |

## Filtering Query Result

- To filter or retrieve specific results, use WHERE Clause
- The WHERE clause applies criteria that each row must meet
- You can use = (equal), >, >=, <, <=, <> (not equal), between, in, like, is, exists operators
- You can use AND, OR and NOT Boolean operators to combine multiple conditions

### Try this

```
SELECT CustomerID, CompanyName, Country, City
FROM Customers WHERE country = 'Finland'
GO
```

- The result

| CustomerID | CompanyName    | Country | City     |
|------------|----------------|---------|----------|
| WARTH      | Wartian Herkku | Finland | Oulu     |
| WILMK      | Wilman Kala    | Finland | Helsinki |

```
SELECT CustomerID, CompanyName, Country, City
FROM Customers
WHERE country = 'Finland' OR country = 'Austria'
GO
```

- The SQL IN condition is used to help reduce the need for multiple OR conditions
- IN requires a set of values and Records that have field value equal to one in the set, are included
- The syntax for the SQL IN condition is:  
expression IN (value1, value2, ..., value\_n)
- The above query can be written as  
SELECT CustomerID, CompanyName, Country, City  
FROM Customers WHERE country IN ('Finland', 'Austria')  
GO
- The result

| CustomerID | CompanyName | Country | City |
|------------|-------------|---------|------|
|------------|-------------|---------|------|

|       |                  |         |          |
|-------|------------------|---------|----------|
| ERNSH | Ernst Handel     | Austria | Graz     |
| PICCO | Piccolo und mehr | Austria | Salzburg |
| WARTH | Wartian Herkku   | Finland | Oulu     |
| WILMK | Wilman Kala      | Finland | Helsinki |

- More filters

### Try this

```
SELECT ProductID, ProductName, UnitPrice, UnitsInStock
FROM Products
WHERE UnitPrice > 100
GO
```

- The result

| ProductID | ProductName             | UnitPrice | UnitsInStock |
|-----------|-------------------------|-----------|--------------|
| 29        | Thüringer Rostbratwurst | 123.79    | 0            |
| 38        | Côte de Blaye           | 263.50    | 17           |

### Try this

```
SELECT ProductID, ProductName, UnitPrice, UnitsInStock
FROM Products
WHERE UnitPrice >= 90 and UnitPrice<=300
GO
```

- The result

| ProductID | ProductName     | UnitPrice | UnitsInStock |
|-----------|-----------------|-----------|--------------|
| 9         | Mishi Kobe Niku | 97.00     | 29           |
| 29        | Thüringer ..    | 123.79    | 0            |
| 38        | Côte de Blaye   | 263.50    | 17           |

- The above query can be written as using BETWEEN operator
- Syntax:
- BETWEEN value1 and value2
- BETWEEN is inclusive - includes margin values

### Try this

```
SELECT ProductID, ProductName, UnitPrice, UnitsInStock
FROM Products
WHERE UnitPrice BETWEEN 90 AND 300
GO
```

## Filtering: Using LIKE operator

- LIKE determines whether a specific character string matches a specified pattern.
- A pattern can include regular characters and wildcard characters.
- SQL wildcards can substitute for one or more characters when searching for data in a database.
- With SQL, the following wildcards can be used:

| Wildcard | Description                            | Example                                                                                                         |
|----------|----------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| %        | Any string of zero or more characters. | <b>WHERE title LIKE '%computer%'</b> finds all book titles with the word 'computer' anywhere in the book title. |
| _        | Any single character.                  | <b>WHERE au_fname LIKE '_ean'</b> finds all four-letter first names that end with ean (Dean, Sean, and so on).  |
| [ ]      | Any single character within            | <b>WHERE au_lname LIKE '[C-P]arsen'</b> finds author                                                            |

|     |                                                     |                                                                                                                          |
|-----|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
|     | the specified range ([a-f]) or set ([abcdef]).      | last names ending with arsen and starting with any single character between C and P, for example Carsen, Larsen, Karsen. |
| [^] | Any single character not within the specified range | WHERE name LIKE 'de[^l]%' all names starting with de and the next letter is not l.                                       |

### Try this

```
SELECT CustomerID, CompanyName, Country
FROM Customers WHERE CustomerID LIKE 'A%'
GO
```

- The query returns all customers whose CustomerID starts with letter A
- The result

| CustomerID | CompanyName                        | Country |
|------------|------------------------------------|---------|
| ALFKI      | Alfreds Futterkiste                | Germany |
| ANATR      | Ana Trujillo Emparedados y helados | Mexico  |
| ANTON      | Antonio Moreno Taquería            | Mexico  |
| AROUT      | Around the Horn                    | UK      |

### Try this

```
SELECT CustomerID, CompanyName, Country
FROM Customers WHERE CustomerID LIKE '%A'
GO
```

- The query returns all customers whose CustomerID ends with letter A
- The result

| CustomerID | CompanyName                          | Country   |
|------------|--------------------------------------|-----------|
| FAMIA      | Familia Arquibaldo                   | Brazil    |
| FISSA      | FISSA Fabrica Inter. Salchichas S.A. | Spain     |
| HILAA      | HILARION-Abastos                     | Venezuela |
| MAGAA      | Magazzini Alimentari ..              | Italy     |
| SAVEA      | Save-a-lot Markets                   | USA       |
| WOLZA      | Wolski Zajazd                        | Poland    |

### Try this

```
SELECT CustomerID, CompanyName, Country
FROM Customers WHERE CustomerID LIKE 'AL_KI'
GO
```

- The query returns the customers whose CustomerID starts with letter AL followed by any character and then KI
- The result

| CustomerID | CompanyName         | Country |
|------------|---------------------|---------|
| ALFKI      | Alfreds Futterkiste | Germany |

### Try this

```
SELECT CustomerID, CompanyName, Country
FROM Customers WHERE CustomerID LIKE 'AL[A-Z]KI'
GO
```

- The query returns the customers whose CustomerID starts with letter AL followed by any character of the set A to Z and then KI
- The result

| CustomerID | CompanyName         | Country |
|------------|---------------------|---------|
| ALFKI      | Alfreds Futterkiste | Germany |

Try this

```
SELECT CustomerID, CompanyName, Country
FROM Customers WHERE Country LIKE '[^A-U]%'
GO
```

- The query returns the customers whose Country does not start with any letter specified in the set A to U.
- The result

| CustomerID | CompanyName          | Country   |
|------------|----------------------|-----------|
| GROSR      | GROSELLA-Restaurante | Venezuela |
| HILAA      | HILARION-Abastos     | Venezuela |
| LILAS      | LILA-Supermercado    | Venezuela |
| LINOD      | LINO-Delicateses     | Venezuela |

## Filtering: Comparing NULL

- The value NULL means the data value for the column is unknown or not available.
- NULL is not equivalent to zero (numeric or binary value), a zero-length string, or blank (character value)
- You cannot operate on NULL
- Any operation on NULL yields to NULL, for example following operations are invalid

→ NULL + somevalue - yields to NULL

→ NULL = somevalue - yields to NULL

- It is not possible to test for NULL values with comparison operators, such as =, <, or <>.
- We will have to use the IS NULL and IS NOT NULL operators instead.
- The following query does not work.

```
SELECT CustomerID, Fax FROM Customers
WHERE fax = NULL
```

- But the following works.

```
SELECT CustomerID, Fax FROM Customers
WHERE fax IS NULL
GO
```

- The result

| CustomerID | Fax  |
|------------|------|
| ANTON      | NULL |
| BSBEV      | NULL |
| CHOPS      | NULL |

## Eliminating Duplicates: DISTINCT

- The DISTINCT keyword eliminates duplicate rows from the results of a SELECT statement.
- For example, if you select all the Countries in Customers without DISTINCT, 91 rows are returned but with DISTINCT keyword 21 rows are returned.
- Compare the following two queries

```
SELECT Country FROM Customers ORDER BY Country
SELECT DISTINCT Country FROM Customers
GO
```

- The result

| Country   |
|-----------|
| Argentina |
| Argentina |

|           |
|-----------|
| Argentina |
| Austria   |
| ...       |

| Country   |
|-----------|
| Argentina |
| Austria   |
| Belgium   |
| Brazil    |
| ...       |

## Calculated column in SELECT

- A calculated column, sometimes called derived column, is one that is dependent on one or more other columns in the same table.

```
SELECT FirstName + ' ' + LastName AS 'Full Name',
BirthDate, HireDate FROM Employees
GO
```

- The Result

| Full Name       | BirthDate                  | HireDate                   |
|-----------------|----------------------------|----------------------------|
| Nancy Davolio   | 1948-12-08<br>00:00:00.000 | 1992-05-01<br>00:00:00.000 |
| Andrew Fuller   | 1952-02-19<br>00:00:00.000 | 1992-08-14<br>00:00:00.000 |
| Janet Leverling | 1963-08-30<br>00:00:00.000 | 1992-04-01<br>00:00:00.000 |
| ...             | ...                        | ...                        |

```
SELECT ProductName, UnitPrice,
UnitPrice*5/100 AS 'Discount',
UnitPrice - UnitPrice*5/100 AS 'Discounted Price'
FROM Products
GO
```

- The result

| ProductName   | UnitPrice | Discount | Discounted Price |
|---------------|-----------|----------|------------------|
| Chai          | 18.00     | 0.90     | 0.90             |
| Chang         | 19.00     | 0.95     | 0.95             |
| Aniseed Syrup | 10.00     | 0.50     | 0.50             |
| ...           | ...       | ...      | ...              |

- You cannot refer a calculated in other calculate column. The following query is invalid.

```
SELECT ProductName, UnitPrice,
UnitPrice*5/100 'Discount',
UnitPrice - Discount AS 'Discounted Price'
FROM Products
```

## Logical test in SELECT: CASE

- Sometimes we need modify column value based on criteria.
- In that case, we can use CASE statement.
- The simple CASE expression compares an expression to a set of simple expressions to determine the result.

```
CASE
WHEN boolean-expression THEN true-result
```



```
WHEN boolean-expression THEN true-result
```

```
...
```

```
ELSE else-result
```

```
END
```

- In this form, true-result is executed when boolean-expression evaluates to True. The ELSE statement is executed if none of the WHEN expressions result in a True value; if there is no ELSE value, the function returns a Null value

Try this

```
SELECT ProductName,
CASE WHEN UnitPrice <=30 THEN 'Inexpensive'
 WHEN UnitPrice <=80 THEN 'Mid-range'
 ELSE 'Expensive'
END AS 'Product Type'
FROM Products
GO
```

- The result

| ProductName                | Product Type |
|----------------------------|--------------|
| Chai                       | Inexpensive  |
| ..                         | ..           |
| Northwoods Cranberry Sauce | Mid-range    |
| Mishi Kobe Niku            | Expensive    |
| ...                        | ...          |

## Aggregate Functions

- Aggregate functions perform a calculation on a set of values and return a single value. Except for COUNT, aggregate functions ignore null values.
- Aggregate functions are frequently used with the GROUP BY clause of the SELECT statement.
- All aggregate functions are deterministic. This means aggregate functions return the same value any time that they are called by using a specific set of input.
- Common aggregate functions are: COUNT, AVG, MAX, MIN, SUM

## Aggregate Functions: COUNT

- COUNT Returns the number of items in a group.
- COUNT (\*) returns the number of rows in a specified table.
- COUNT (expression) evaluates expression for each row and counts non-null values.
- COUNT (ALL expression) same as COUNT (expression).
- COUNT (DISTINCT expression) evaluates expression for each row and counts unique, non-null values.

Try this

```
SELECT COUNT(*) FROM Customers
SELECT COUNT(CustomerID) FROM Customers
SELECT COUNT(Fax) FROM Customers
GO
```

- The result

| (No column name) |
|------------------|
| 91               |

| (No column name) |
|------------------|
| 91               |

(No column name)

69

- COUNT(\*) – returns number of rows
- COUNT(Column) –does not include null
- The CustomerID has no nulls but Fax has so results are different.

Try this

```
SELECT COUNT(Country) FROM Customers
SELECT COUNT(DISTINCT Country) FROM Customers
GO
```

- The result

(No column name)

91

(No column name)

21

- Count (expression) includes duplicate values but if you use DISTINCT before expression only unique values are counted
- There is COUNT\_BIG aggregate taht works like the COUNT function. The only difference between the two functions is their return values. COUNT always returns an int data type value. COUNT\_BIG always returns a bigint data type.

## Aggregate with Grouping

- You cannot use aggregates with non-aggregates unless there is grouping.
- The following will produce error.  
SELECT Country, COUNT (CustomerID) 'No of Customers'  
FROM Customers

- The rule is: If there is non-aggregate column with aggregates, each non-aggregate column must be listed with GROUP BY clause.

```
SELECT Country, COUNT(CustomerID) 'Count'
FROM Customers
GROUP BY Country
ORDER BY Country
GO
```

- The result

| Country   | No of Customers |
|-----------|-----------------|
| Argentina | 3               |
| Austria   | 2               |
| Belgium   | 2               |
| Brazil    | 9               |
| Canada    | 3               |
| ...       | ...             |

```
SELECT Country, City, COUNT(CustomerID) 'Count'
FROM Customers
GROUP BY Country, City
ORDER BY Country
GO
```

- The result

| Country   | City         | No of Customers |
|-----------|--------------|-----------------|
| Argentina | Buenos Aires | 3               |
| Austria   | Graz         | 1               |

|         |           |     |
|---------|-----------|-----|
| Austria | Salzburg  | 1   |
| Belgium | Bruxelles | 1   |
| Belgium | Charleroi | 1   |
| ...     | ...       | ... |

## Grouping with filter

- To apply filter on a column that is present in GROUP BY list, we use having clause.

### Try this

```
SELECT Country, COUNT(CustomerID) AS 'Count'
FROM Customers
GROUP BY Country
HAVING Country = 'Brazil'
GO
```

- The result

| Country | Count |
|---------|-------|
| Brazil  | 9     |

- You can use WHERE in this case but it must be placed before GROUP BY clause.

```
SELECT Country, COUNT(CustomerID) AS 'Count'
FROM Customers
WHERE Country = 'Brazil'
GROUP BY Country
GO
```

- The result

| Country | Count |
|---------|-------|
| Brazil  | 9     |

- To apply filter on a column that is not present in GROUP BY list, we use WHERE clause.

```
SELECT Country,
COUNT (CustomerID) AS 'No of Customers'
FROM Customers
WHERE City = 'Buenos Aires'
GROUP BY Country
GO
```

- The result

| Country   | Count |
|-----------|-------|
| Argentina | 3     |

## Aggregate: MAX, MIN

- MAX returns the maximum value in the expression.
- MIN returns the minimum value in the expression.
- Syntax:  
MAX (expression)  
MIN (expression)

### Try this

```
SELECT MAX (UnitPrice) 'Highest Price',
MIN (UnitPrice) 'Lowest Price'
FROM Products
GO
SELECT MAX (OrderDate) 'Last Order Placed On',
MIN (OrderDate) 'First Order Placed On'
FROM Orders
GO
```

- The result

| Highest Price | Lowest Price |
|---------------|--------------|
| 263.50        | 2.50         |

| Last Order Placed On    | First Order Placed On   |
|-------------------------|-------------------------|
| 1998-05-06 00:00:00.000 | 1996-07-04 00:00:00.000 |

```
SELECT CustomerID, MIN (OrderDate) 'First Order Date'
FROM ORDERS
GROUP BY CustomerID
HAVING CustomerID='ALFKI'
GO
```

- The result

| CustomerID | First Order Date        |
|------------|-------------------------|
| ALFKI      | 1997-08-25 00:00:00.000 |

## Aggregate: SUM, AVG

- SUM returns the sum of all the values in the expression. SUM can be used with numeric columns only. Null values are ignored.
- AVG returns the average of the values in a group. Null values are ignored.

### Try this

```
SELECT SUM (UnitPrice), AVG (UnitPrice)
FROM Products
GO
```

- The result

| (No column name) | (No column name) |
|------------------|------------------|
| 2222.71          | 28.8663          |

```
SELECT OrderID,
SUM(UnitPrice*Quantity*(1 - Discount)) 'Order Value'
FROM [Order Details]
GROUP BY OrderID
GO
```

- The Result

| OrderID | Order Value      |
|---------|------------------|
| 10248   | 440              |
| 10249   | 1863.39999389648 |
| 10252   | 3597.89990234375 |
| ...     | ...              |

## SELECT...INTO Statement

- SELECT...INTO creates a new table and inserts the resulting rows from the query into it.
- Syntax:  
SELECT Column\_List  
INTO new\_table  
FROM source
- The new table to be created, based on the columns in the select list.
- Each column in new table has the same name, data type, nullability, and value as the corresponding expression in the select list.

```
SELECT CompanyName, Phone
INTO CustContact_Finland
FROM Customers c
WHERE Country ='Finland'
GO
SELECT * FROM CustContact_Finland
GO
```

| CompanyName    | Phone       |
|----------------|-------------|
| Wartian Herkku | 981-443655  |
| Wilman Kala    | 90-224 8858 |

## Union

- It allows you to join two query results into one

### Try this

```
SELECT City FROM Customers
```

```
UNION
SELECT City FROM Employees
GO
```

- The result  
City

```

Aachen
Albuquerque
Anchorage
...
```

(69 row(s) affected)

- Union by default removes duplicates
- To include duplicates use ALL option

### Try this

```
SELECT City FROM Customers
UNION ALL
SELECT City FROM Employees
GO
```

- The result  
City

```

Aachen
Albuquerque
Anchorage
...
```

(182 row(s) affected)

-